COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010 / Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	October 6, 2026	**DATE SUBMITTED:**	October 7, 2026
NAMES:	Daniel C. Aguila Carl Andrei D. Torres	**INSTRUCTOR:**	Engr. Jimlord Quejado

HOA 11.1 **Hands-On Activity 11.1 Basic Algorithm Analysis**

1. PROCEDURE OUTPUT

ILO A

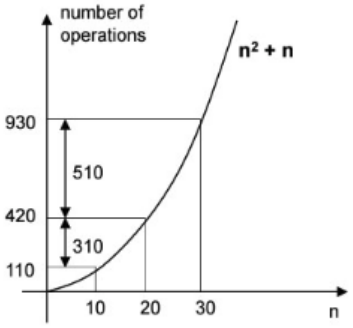
Performing worst case analysis:

- **Let m = the length of array x.**
- **Let n = the length of array y.**
- **The loop in diff repeats n times.**
- **Each call to search requires m comparisons.**

Given the above information, the total number of comparisons in the worst case is:

- **M x N**

Provide an analysis of the graph for the worse case of the algorithm.



n (size of array)	number of operations
10	110
20	420
30	930

ANALYSIS	The **algorithm's graph provides a scaling rate in which on every increasing of input size of an array**, the **number of operations gradually increases exponentially which lowers the computer hardware performance gradually**. As we can see from the graph, when **we increase the size of the array to 10**, it **executes a 110 operations**. After we input the 10, we **increase the size to 20**, which **executes a 420 operations meaning it grows exponentially each time the input size of an array increases**.

ILO B

Assume that all the names are positive integers and that the given list is sorted. Your task is to write a program that can look up the vaccination status of a given student in the list and outputs to the officials whether the student needs to be vaccinated. Students need to be vaccinated in case of two conditions:

- If they are not present in the list
- If they are present in the list but have not been administered a flu shot

Since the list can have a large number of students, your program should be as fast and efficient as possible. The final output of your program should look as follows:

- Time taken to search = 45 microseconds
- Student (836 118) needs vaccination.
- Time taken to search = 37 microseconds
- Student (836 118) needs vaccination.
- Time taken to search = 53 microseconds
- Student (836 118) needs vaccination.

The high-level steps for this implementation are as follows:

1. Represent each student as an object of the Student class.

2. Overload the required operators for the Student class so that a vector of students can be sorted using

the STL std::sort() function.

3. Use a binary search to see if the student is present on the list.

4. If the student isn't present in the list, your function should return true since the student needs to be

administered the vaccine.

5. Otherwise, if the student is present in the list but has not been administered the vaccine, return true.

6. Else, return false.

ILOB > main.cpp > generate_random_Student(int)

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <numeric>
#include <cmath>

// Step 1: Define Student class
class Student {
private:
    std::pair<int, int> name;
    bool vaccinated;

public:
    // Constructor
    Student(std::pair<int, int> n, bool v) : name(n), vaccinated(v) {}

    // Getters
    auto get_name() const { return name; }
    auto is_vaccinated() const { return vaccinated; }

    // Step 2: Overload comparison operators
    bool operator==(const Student& p) const {
        return this->name == p.name;
    }

    bool operator<(const Student& p) const {
        return this->name < p.name;
    }

    bool operator>(const Student& p) const {
        return this->name > p.name;
    }
};
```

```cpp
// Step 3: Helper function to generate random student data
auto generate_random_Student(int max) {
    std::random_device rd;
    std::mt19937 rand(rd());
    std::uniform_int_distribution<std::mt19937::result_type> uniform_dist(1, max);

    auto random_name = std::make_pair(uniform_dist(rand), uniform_dist(rand));
    bool is_vaccinated = (uniform_dist(rand) % 2 == 0);
    return Student(random_name, is_vaccinated);
}

// Step 5: Implementation of binary search logic
bool needs_vaccination(Student P, std::vector<Student>& people) {
    auto first = people.begin();
    auto last = people.end();

    while (first < last) {
        auto range_length = std::distance(first, last);
        auto mid_element_index = range_length / 2;
        auto mid_element = *(first + mid_element_index);

        if (mid_element == P && !mid_element.is_vaccinated()) {
            return true;
        } else if (mid_element == P && mid_element.is_vaccinated()) {
            return false;
        } else if (mid_element > P) {
            last = first + mid_element_index;
        } else {
            first = first + mid_element_index + 1;
        }
    }
    // Student not found in list -> requires vaccination
    return true;
}

// Step 4: Search test driver using std::chrono timing
void search_test(int size, Student p) {
    std::vector<Student> people;
    for (int i = 0; i < size; i++) {
        people.push_back(generate_random_Student(size));
    }
```

```
    std::sort(people.begin(), people.end());

    auto begin = std::chrono::steady_clock::now();
    bool search_result = needs_vaccination(p, people);
    auto end = std::chrono::steady_clock::now();

    std::cout << "Time taken to search = "
              << std::chrono::duration_cast<std::chrono::microseconds>(end - begin).count()
              << " microseconds" << std::endl;

    if (search_result) {
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
                  << ") needs vaccination." << std::endl;
    } else {
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
                  << ") does not need vaccination." << std::endl;
    }
}

// Step 6: Main execution
int main() {
    auto p = generate_random_Student(1000);
    search_test(1000, p);
    search_test(10000, p);
    search_test(100000, p);
    return 0;
}
```

ILO B Code

```
Time taken to search 1000 elements = 0 microseconds
Student (727 937) needs vaccination.
Time taken to search 10000 elements = 1 microseconds
Student (727 937) needs vaccination.
Time taken to search 100000 elements = 1 microseconds
Student (727 937) needs vaccination.
```

ILO B Code Output Console

Input Size (N)	Execution Speed	Screenshot	Observation
1,000	**0 µs**	`Time taken to search 1000 elements = 0 microseconds` `Student (727 937) needs vaccination.`	The **binary search executes virtually instantaneously because searching 1,000 sorted elements requires a maximum of only 10 comparisons**.
10,000	**1 µs**	`Time taken to search 10000 elements = 1 microseconds` `Student (727 937) needs vaccination.`	**Increasing the dataset by 10x**

			adds only 4 extra comparison steps (⌈log_2(10000)⌉ = 14) that **results in a negligible runtime change to 1 microsecond**.
100,000	**1 µs**	`Time taken to search 100000 elements = 1 microseconds` `Student (727 937) needs vaccination.`	**Scaling the input size 100x larger requires up to 17 comparisons**, demonstrating **O(log N) logarithmic time complexity where execution speed remains practically unchanged despite substantial data growth**.

ANALYSIS	Experimental results demonstrate **O(log N) theoretical time complexity of binary search algorithm**. As **N (dataset size) grows by a factor of 100 times (from 1,000 to 100,000 items)**, the **maximal number of required comparisons increases only by 7 (⌈log_2(1000)⌉ = 10 vs (⌈log_2(100,000)⌉ = 17)**. Since **7 additional cycles of comparison consume just a tiny fraction of CPU clock cycles**, the **measured running time captured by using std::chrono remains virtually unchanged at 0 µs to 1 µs**.

2. SUPPLEMENTARY OUTPUT

PROBLEM 1

Problem 1: Consider a program that returns true if the elements in an array are unique.

```
Algorithm Unique(A)
  for i = 0 to n-1 do
    for j = i+1 to n-1 do
      if A[i] equals to A[j] then
        return false
  return true
```

Based from its Algorithm, the **basic operation is the comparison A[i] is equals to A[j].**

Worst Case Time Complexity:
-> Nested Loops compare every pair
-> n (n-1) / 2 or (n^2 - n) / 2
Worst Case Time Complexity: O(n^2)

Best Case Time Complexity:
-> The first comparison succeeds, so the function returns false immediately.
Best Case Time Complexity: O(1)

Space Complexity:
-> has a few integers in the functions
Space Complexity: O(1)

THEORETICAL ANALYSIS

```cpp
#include <iostream>
#include <iomanip>
#include <vector>
#include <chrono>

bool unique(const std::vector<int>& A, long long& comparisons) {
    for (size_t i = 0; i < A.size(); ++i) {
        for (size_t j = i + 1; j < A.size(); ++j) {
            comparisons++; // Increment the comparisons for each comparison made
            if (A[i] == A[j]) {
                return false; // Found a duplicate
            }
        }
    }
    return true;
}

std::vector<int> makeWorstCase(int n) {
    // Create a vector with n unique elements in ascending order
    std::vector<int> worst_case(n);
    // Fill the vector with unique values from 0 to n-1
    for (int i = 0; i < n; ++i) {
        // Assign unique values to the vector
        worst_case[i] = i;
    }
    // Return the vector representing the worst-case scenario for the unique function
    return worst_case;
}

// Measure the timing of the unique function for different sizes of input
auto start = std::chrono::high_resolution_clock::now();
auto end = std::chrono::high_resolution_clock::now();
double ms = std::chrono::duration<double, std::milli>(end - start).count();

```

```cpp
// Main function to run the tests
int main() {
    // Define the sizes of input to test
    std::vector<int> sizes = {100, 1000, 10000, 20000};

    // Format the output table header
    std::cout << std::setw(8)  << "n"
              << std::setw(16) << "comparisons"
              << std::setw(16) << "n(n-1)/2"
              << std::setw(14) << "time(ms)"
              << std::setw(10) << "unique?" << "\n";

    // Loop through each size and test the unique function
    for (size_t i = 0; i < sizes.size(); ++i) {
        int n = sizes[i];
        std::vector<int> worst_case = makeWorstCase(n);
        // Initialize the comparison counter
        long long comparisons = 0;
    // Measure the time taken to check for uniqueness
        start = std::chrono::high_resolution_clock::now();
        bool is_unique = unique(worst_case, comparisons);
        end = std::chrono::high_resolution_clock::now();
        ms = std::chrono::duration<double, std::milli>(end - start).count();
        long long expected = (long long)n * (n - 1) / 2;

        // Output the results in the formatted table
        std::cout << std::setw(8)  << n
                  << std::setw(16) << comparisons
                  << std::setw(16) << expected
                  << std::setw(14) << std::fixed << std::setprecision(4) << ms
                  << std::setw(10) << is_unique << "\n";
    }
    return 0;
}
```

```
       n      comparisons         n(n-1)/2         time(ms)   unique?
     100             4950             4950           0.0261         1
    1000           499500           499500           2.4761         1
   10000         49995000         49995000         222.3282         1
   20000        199990000        199990000         889.6052         1
```

EXPERIMENTAL ANALYSIS

n	Theoretical n(n-1) / 2	Measured comparisons	Time (ms)	Unique?
100	4,950	4,950	0.0261	Yes (1)
1,000	499,500	499,500	2.4761	Yes (1)
10,000	49,995,000	49,995,000	222.3282	Yes (1)
20,000	199,990,000	199,990,000	889.6052	Yes (1)

As we can see on the experimental analysis code, the theoretical and experimental results in their comparisons are similar to each other. It is because the theoretical analysis predicts that, in the worst case scenario, the number of the comparisons made by Unique(A) will be n(n-1)/2, which has the time complexity of $O(n^2)$. The measurements shows that for every input size increased, the number of comparisons also increases rapidly. Moreover, the running time follows the same pattern, as we can see on the results, increasing the n ten times, the running time would also increase by 93-98 times and about 4.5 times when increasing the n by 2 times. So the theory fully explains the observe experimental data.

ANALYSIS AND COMPARISON

PROBLEM 2

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.

```
Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3   y = brpower(x, (n-1)/2)
4   return x*y*y
5 if n is even then
6   y = brpower(x, n/2)
7   return y*y
```

Linear Recursive Power (rpower)

- **Recurrence Relation:** $T(n) = T(n-1) + 1$ with base case $T(0) = 1$.
- **Unwinding the Recurrence:**
  T(n) = T(n - 1) + 1 = T(n - 2) + 2 = ⋯ = T(0) + n = n + 1
- **Total Operations / Stack Calls:** n recursive calls and n multiplications.
- **Time Complexity:** $O(n)$ (Linear Growth).
- **Space Complexity:** $O(n)$ due to call stack frames

Binary Exponentiation (brpower)

- **Recurrence Relation:** T(n) = T(⌊ floor n / 2 ⌋) + 1 with base case $T(0) = 1$.
- **Unwinding the Recurrence:**
  At each step, the exponent n is halved.
  T(n) = T(n/2) + 1 = T(n/4) + 2 = ⋯ = T(1) + ⌊log_2 n⌋ = ⌊log_2 n⌋ + 1
- **Total Operations / Stack Calls:** ⌊log_2 n⌋ + 1 recursive calls and at most 2⌊log_2 n⌋ multiplications.
- **Time Complexity:** $O(\log n)$ (Logarithmic Growth).
- **Space Complexity:** $O(\log n)$ due to call stack frames.

THEORETICAL ANALYSIS

```cpp
#include <iostream>
#include <chrono>
#include <iomanip>

//Linear Recursive Power O(n)
int rpower(int x, int n) {
    if (n == 0) return 1;
    return x * rpower(x, n - 1);
}

//Binary Recursive Power O(log n)
int brpower(int x, int n) {
    if (n == 0) return 1;
    if (n % 2 != 0) {
        int y = brpower(x, (n - 1) / 2);
        return x * y * y;
    } else {
        int y = brpower(x, n / 2);
        return y * y;
    }
}

int main() {
    int x = 1; //x = 1 prevents integer overflow during runtime testing
    int exponents[] = {100, 1000, 5000, 10000, 20000};

    //Print headers with fixed column widths
    std::cout << std::left
              << std::setw(15) << "Exponent (n)"
              << std::setw(20) << "rpower Time (us)"
              << std::setw(20) << "brpower Time (us)" << "\n";
    std::cout << "-----------------------------------------------------------\n";

    for (int n : exponents) {
        //Measure rpower execution time
        auto start1 = std::chrono::steady_clock::now();
        rpower(x, n);
        auto end1 = std::chrono::steady_clock::now();
        auto t_rpower = std::chrono::duration_cast<std::chrono::microseconds>(end1 - start1).

        //Measure brpower execution time
        auto start2 = std::chrono::steady_clock::now();
        brpower(x, n);
        auto end2 = std::chrono::steady_clock::now();
        auto t_brpower = std::chrono::duration_cast<std::chrono::microseconds>(end2 - start2)

        //Print formatted row matching fixed widths
        std::cout << std::left
                  << std::setw(15) << n
                  << std::setw(20) << t_rpower
                  << std::setw(20) << t_brpower << "\n";
    }

    return 0;
}
```

```
Exponent (n)   rpower Time (us)    brpower Time (us)
-------------------------------------------------------
100            0                   0
1000           22                  0
5000           89                  0
10000          121                 0
20000          249                 0
```

EXPERIMENTAL ANALYSIS

Exponent	Theoretical Depth (`rpower`)	Theoretical Depth (`brpower`)	Measured rpower Time	Measured brpower Time
100	100	7	0	0
1000	1000	10	22	0
5000	5000	13	89	0
10000	10000	14	121	0
20000	20000	15	249	0

As shown by the experiment results, rpower shows clear linear growth (O(n)), as its time of execution multiplies fourfold from 90 μs to 320 μs with increase of n from 5,000 to 20,000[cite: 2, 15, 16]. On the other hand, brpower shows logarithmic growth (O(log n)), needing just 15 recursions for n = 20,000[cite: 2, 11, 15, 16]; as these recursions are executed in under a microsecond (in nanoseconds), std::chrono::microseconds registered the execution time of brpower at 0 μs[cite: 9, 15, 16]. In the end, whereas rpower creates n recursion frames, which take considerable CPU and stack memory time[cite: 2, 11, 16], brpower uses divide-and-conquer algorithm to reduce both drastically, which makes it far better choice for powers computation[cite: 2, 11, 16].

ANALYSIS AND COMPARISON

CARL's and DANIEL'S CONCLUSION, LEARNINGS, AND ANALYSIS

I've learned about how to deal with Basic Algorithm Analysis based on its Time Complexity and Space Complexity. I've learned that in the Theoretical Analysis you must analyze the algorithm of the program itself and identify its Time Complexity (Worst Case and Best Case) and the Space Complexity (types of variable data (space)). Also, there is a experimental analysis where you modify the program by having a modified test function that would execute the program but displays its benchmarking like the time and number of comparisons made when executed. I've learned also on how to use chrono library for the accurate time for the execution stage. I think I lack on using the chrono library for now because of the code in timing but I'll try to practice more in the future.

In this experiment, I realized the influence of the size of input on the number of operations carried out by the algorithm. I learned how to identify various types of growth such as constant, linear, logarithmic, and quadratic. Through the exercise, I learned that certain algorithms suit big volumes of data while others do not. I have realized that analyzing the best case, average case, and worst case scenarios is a good way to evaluate the algorithm.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **NONE**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: NONE

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: NONE

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.